// GUIDA GRATIS · TUTORIAL PASSO PASSO

IL CORSO DI AI *engineering gratis.*

Questa guida ti porta, dal primo comando, da zero a una lezione del corso **AI Engineering from Scratch** eseguita sul tuo computer. Ogni passo ha il comando esatto, quello che devi vedere (output copiato da un'esecuzione vera, fatta il 28/10/2026 su un Mac) e gli errori più comuni. Alla fine puoi rifare tutto da solo, senza guardare il video.

Cosa ottieni	un corso con lezioni e codice, gratis	**542 lezioni · 20 fasi**
Cosa ti serve	Python, Git, un terminale	**~15 minuti**
Licenza	puoi usarlo e copiarlo	**MIT**
Lingue del codice	nelle lezioni	**Python · TypeScript · Rust · Julia**
Autore	repository di terzi, non nostro	**rohitg00**

// LA MAPPA DI QUESTA GUIDA

00 controlla gli strumenti · **01** guarda il corso sul sito · **02** scegli da dove partire · **03** scarica il repository · **04** verifica l'ambiente · **05** prima lezione · **06** seconda lezione · **07** il tutor AI · **08** metodo di studio · verifica finale.

// ONESTÀ SUI DATI

Stelle (75.134), fork (13.194) e licenza sono letti dall'API pubblica di GitHub il 28/10/2026; lezioni e fasi dal README. Cambiano nel tempo. Le ore indicate sono stime dell'autore, non misurate da noi.

// PASSO 00 · PRIMA DI INIZIARE

00 CONTROLLA GLI *strumenti*

servono tre cose: Python 3.11 o più nuovo, Git, un terminale

01 **Apri il terminale.**
Su Mac: Spotlight (Cmd+Spazio), scrivi “Terminale”, Invio. Su Windows usa PowerShell o WSL. Su Linux il terminale della tua distribuzione.

02 **Controlla le versioni.**
Copia i tre comandi qui sotto uno alla volta e premi Invio. Ognuno stampa una versione: se stampa “command not found”, quello strumento manca.

03 **Installa quello che manca.**
Python da python.org, Git da git-scm.com. Node.js serve solo per il tutor AI del passo 07 e puoi saltarlo per ora.

```
python3 --version
git --version
node --version
```

```
OUTPUT VERO · IL MIO MAC, 28/10/2026
Python 3.15.0
git version 2.55.0
v26.6.0
```

Python	deve essere 3.11 o più nuovo	obbligatorio
Git	per scaricare il repository	obbligatorio
Node.js	solo per il tutor AI (passo 07)	facoltativo
Un editor	per leggere il codice, es. VS Code	consigliato

// ERRORI COMUNI

`python3: command not found` → su Windows prova `python --version` o `py --version`. `Python 3.9` o più vecchio → il controllo del passo 04 fallirà: aggiorna Python prima di andare avanti.

// PASSO 01 · GUARDA IL CORSO SUL SITO

01 PRIMA GUARDA *dove vai*

aiengineeringfromscratch.com · nessuna installazione, nessun download



SCREENSHOT VERO · LEZIONE “DEV ENVIRONMENT” SUL SITO DEL CORSO · 28/10/2026

01 Apri il link della prima lezione.
aiengineeringfromscratch.com/lesson?path=phases/00-setup-and-tooling/01-dev-environment

02 Guarda la struttura.
A sinistra le lezioni della fase, a destra l'indice della pagina: Il problema, Il concetto, Costruiscilo (a passi), Usalo, Consegnalo, Esercizi.

03 Nota il riquadro “Act on this lesson”.
Ogni lezione ti chiede: leggi, costruisci, esegui, dimostra con un output, continua. È il metodo che seguiremo in questa guida.

// A COSA SERVE QUESTO PASSO

Capire che forma ha una lezione prima di installare qualsiasi cosa. Se il sito ti basta, puoi studiare tutto da lì: il README dice che il percorso sul sito non richiede setup.

// PASSO 02 · SCEGLI DA DOVE PARTIRE

02 NON PARTIRE *dal fondo*

non serve fare le 20 fasi in ordine: scegli la riga che ti somiglia

Nuovo a programmazione e AI	Fase 0 · Setup	~306 ore
Sai Python, nuovo al machine learning	Fase 1 · Matematica	~270 ore
Sai ML, nuovo al deep learning	Fase 3 · Deep Learning	~200 ore
Sai deep learning, vuoi LLM e agenti	Fase 10 · LLM da zero	~100 ore
Sei sviluppatore, vuoi solo gli agenti	Fase 14 · Agent Engineering	~60 ore
Solo MCP (Model Context Protocol)	percorso dedicato	~23 ore
Solo Agent Skills	percorso dedicato	~9,5 ore
Solo reverse engineering	percorso dedicato	~32,5 ore

01 **Scegli UNA riga.**
Se esiti tra due, prendi quella con più ore: costa solo qualche lezione in più, e non ti perdi basi.

02 **Fai il test di 10 minuti.**
Apri la prima lezione della fase scelta: se non capisci i termini, scendi di una riga; se è tutto ovvio, sali.

03 **Le ore sono stime.**
Sono quelle dichiarate dall'autore nel README. Per una persona che studia 1 ora al giorno, 100 ore sono circa 3 mesi: pianifica con questa misura, non con l'entusiasmo.

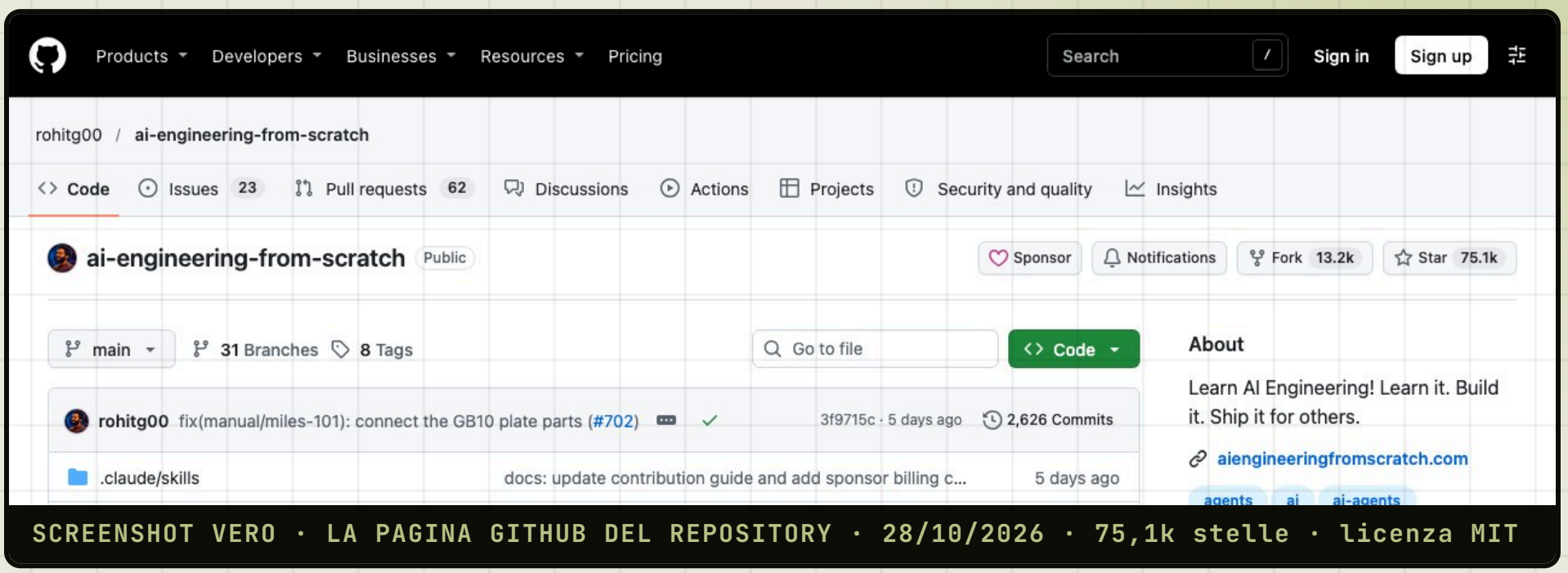
// ERRORE COMUNE

Partire dalla Fase 14 (agenti) senza basi: i concetti sembrano magia e ti blocchi. Se i primi esempi ti sembrano incomprensibili, torna indietro di una fase.

// PASSO 03 · SCARICA IL REPOSITORY

03 PORTA IL CORSO *sul tuo pc*

github.com/rohitg00/ai-engineering-from-scratch



SCREENSHOT VERO · LA PAGINA GITHUB DEL REPOSITORY · 28/10/2026 · 75,1k stelle · licenza MIT

01 **Vai in una cartella ordinata.**
Per esempio la cartella Documenti: **cd ~/Documents** (su Windows **cd $HOME\Documents**).

02 **Clona il repository.**
“Clonare” = scaricare una copia completa del progetto. Il comando crea la cartella ai-engineering-from-scratch.

03 **Entra nella cartella.**
Tutti i comandi delle lezioni partono da qui: è la cartella che contiene README.md e phases/.

```
git clone https://github.com/rohitg00/ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
ls
```

// ERRORI COMUNI

`git: command not found` → installa Git (passo 00). `Could not resolve host` → controlla la connessione. Il download è grande (il repository ha oltre 2.600 commit): se è lento, aggiungi `--depth 1` dopo “clone” per scaricare solo l'ultima versione, come ho fatto io per le prove.

// COSA DEVI VEDERE

Dopo `ls` la cartella mostra, tra le altre, `phases`, `skills`, `README.md`, `LICENSE`. Se non le vedi, sei nella cartella sbagliata.

// PASSO 04 · VERIFICA L'AMBIENTE

04 UN COMANDO, *due spunte*

lo script del corso controlla se sei pronto e ti dice come correggere

01 **Lancia il controllo, dalla cartella del repository.**
Il parametro **--route beginner** controlla solo ciò che serve per iniziare il corso base.

02 **Leggi le righe [PASS] e [FAIL].**
Ogni requisito obbligatorio è PASS o FAIL. Per ogni FAIL lo script scrive il motivo e il comando per correggere (lo dice il README).

03 **Vai avanti solo con "2/2 required checks passed".**
Gli altri strumenti (Node, Rust, GPU…) li controlla più avanti, quando servono.

```
python3 phases/00-setup-and-tooling/01-dev-environment/code/verify.py --route be
ginner
```

OUTPUT VERO · ESEGUITO IL 28/10/2026

```
=== AI Engineering from Scratch: Environment Check ===

Route: Beginner course (`--route beginner`)

  [PASS] Python 3.11+ (required now)
         Python 3.15.0 at /opt/homebrew/opt/python@3.15/bin/python3.15
  [PASS] Git (required now)
         git version 2.55.0 at /opt/homebrew/bin/git

Later checks skipped: 9 tools are not needed to start. Add `--show-later` when you want to inspect
them.

Result: 2/2 required checks passed
Ready to start Beginner course.
Next: python3 phases/01-math-foundations/01-linear-algebra-intuition/code/vectors.py
```

// ERRORI COMUNI

`No such file or directory` → non sei nella cartella del repository: torna al passo 03 e fai **`cd ai-engineering-from-scratch`**. **`[FAIL] Python 3.11+`** → aggiorna Python e rilancia. Su Windows, se **`python3`** non esiste, usa **`python`** al suo posto.

// PERCHÉ FARLO

Se salti questo passo e qualcosa non funziona più avanti, non sai se è colpa tua, dell'ambiente o della lezione. Il controllo toglie il dubbio.

05 ESEGUI IL *primo codice*

Fase 1 · Matematica · lezione 1 · intuizione dell'algebra lineare

01 Lancia lo script.
Usa solo Python, senza librerie da installare. Stampa a schermo vettori, matrici, angoli, proiezioni e rango.

02 Leggi l'output dall'alto in basso.
Ogni blocco "=== Titolo ===" è una lezione in miniatura: somma di vettori, rotazione di 90°, ortogonalità, rango.

03 Fermati sull'ultimo blocco.
Mostra che un livello di rete neurale è una matrice per un vettore: è lì che l'AI "si vede".

```
python3 phases/01-math-foundations/01-linear-algebra-intuition/code/vectors.py
```

OUTPUT VERO · COMPLETO · 28/10/2026

```
=== Vectors ===
a = Vector([1, 2, 3])
b = Vector([4, 5, 6])
a + b = Vector([5, 7, 9])
a - b = Vector([-3, -3, -3])
a * 3 = Vector([3, 6, 9])
a · b = 32
|a| = 3.7417
â (normalized) = Vector([0.2672612419124244, 0.5345224838248488, 0.8017837257372732])
cosine_similarity(a, b) = 0.9746

=== Matrices ===
Rotate Vector([3, 1]) by 90° → Vector([-1, 3])

[… angoli, proiezioni, indipendenza lineare, Gram-Schmidt …]

=== Matrix Rank ===
Identity 2x2 rank: 2
[[1,2],[2,4]] rank: 1
[[1,0,0],[0,1,0]] rank: 2

=== Neural Network Layer (Matrix x Vector) ===
Input (3D):  Vector([1.0, 0.5, -0.3])
Output (2D): Vector([-0.019714737127338927, 0.10873956075097067])
^ This is literally what a neural network layer does.
```

// COSA SIGNIFICA L'ULTIMA RIGA

`Input (3D)` è una lista di tre numeri; **`Output (2D)`** sono due numeri. Il livello li ha trasformati moltiplicandoli per una matrice. Una rete neurale è una pila di trasformazioni così.

06 CAMBIA *un numero*

Fase 1 · lezione 8 · ottimizzazione · gradient descent, momentum, Adam

01 **Lancia il confronto.**
Tre metodi di ottimizzazione cercano lo stesso punto (1, 1) di una funzione difficile. Vedi quanti passi servono a ciascuno.

02 **Confronta le righe a 5000 passi.**
Il gradient descent semplice è ancora lontano (loss 0,04), il momentum è a 0,0036, Adam è praticamente a 0.

03 **Fai la tua prova.**
Apri optimizers.py, cambia un valore (per esempio il learning rate) e rilancia: se non cambia niente, non hai salvato il file.

```
python3 phases/01-math-foundations/08-optimization/code/optimizers.py
```

ESTRATTO DALL'OUTPUT VERO · RIGHE RIFORMATTATE DA NOI

Metodo	Step	Loss
Gradient Descent	0	4.00000000
Gradient Descent	1000	2.09735727
Gradient Descent	5000	0.04083385
SGD + Momentum	5000	0.00355685
Adam	1000	0.00000161
Adam	5000	0.00000000

OUTPUT VERO · ESPERIMENTO “MOMENTUM EFFECT ON SGD”

Beta	Final x	Final y	Loss
0.00	-0.446368	0.206459	2.09718513
0.50	0.363386	0.129014	0.40619883
0.90	0.940412	0.884127	0.00355685
0.99	1.000000	1.000000	0.00000000

// CHE COSA HAI IMPARATO

Più il parametro beta del momentum cresce (0 → 0,99), più la loss finale scende (2,09 → 0,00): lo stesso problema, risolto in modo molto diverso cambiando un solo numero. È il tipo di intuizione che il corso costruisce.

// ERRORE COMUNE

Lo script stampa molte righe: non devi capirle tutte. Guarda solo la colonna **Loss** e controlla che scenda.

// PASSO 07 · FACOLTATIVO · IL TUTOR AI

07 UN TUTOR *nel tuo agente*

`serve Node.js + un agente di codice (es. Claude Code, Codex, Cursor)`

01 **Installa le skill del corso.**
Un solo comando. L'installer ti chiede su quale agente installarle e dove (progetto o utente).

02 **Avvia il tutor.**
In Claude Code scrivi **/start-learning**; in Codex **start-learning**. Un quiz di dieci domande capisce il tuo livello.

03 **Ottieni un piano.**
Salva un piano personalizzato in un file **LEARNING.md** nella cartella dove lavori. Poi **/learn** ti insegna una lezione per volta: concetto, matematica, codice, quiz.

```
npx skills add rohitg00/ai-engineering-from-scratch
```

`/start-learning`	quiz di 10 domande e piano	`LEARNING.md`
`/learn`	una lezione per sessione	`tutor`
`/course-guide`	ti porta alla lezione giusta se sei bloccato	`router`

COSA NON ABBIAMO VERIFICATO NOI

Questo passo è descritto dal README del repository: non abbiamo eseguito l'installer (è interattivo e scrive nel tuo agente). Per questo qui non c'è un output: controlla tu che la cartella scelta, per esempio `.claude/skills/`, contenga le skill dopo l'installazione.

// ERRORI COMUNI

`npx: command not found` → installa Node.js. Il comando **`/start-learning`** non compare → l'agente non vede la cartella delle skill: riavvialo o reinstalla scegliendo la cartella giusta. Se non hai un agente di codice, salta questo passo: i passi 01–06 bastano.

// NON È UN INSEGNANTE UMANO

Non rilascia certificati e non corregge i tuoi progetti. Il valore sta nel costruire le cose: se il tutor ti dà la risposta, prova prima da solo.

// VERIFICA FINALE

FUNZIONA?
Controlla qui.

- ☐ Ho Python 3.11+, Git e un terminale (passo 00).
- ☐ Ho scelto da dove partire e so quante ore mi servono (passo 02).
- ☐ Ho clonato il repository e *ls* mostra *phases* (passo 03).
- ☐ Il controllo ambiente dice *2/2 required checks passed* (passo 04).
- ☐ *vectors.py* finisce con “This is literally what a neural network layer does.” (passo 05).
- ☐ Ho cambiato un numero in *optimizers.py* e ho visto l'output cambiare (passo 06).

Struttura di ogni lezione	code/ · docs/en.md · outputs/	**stessa ovunque**
Lingua delle lezioni	il testo è in inglese	**docs/en.md**
Traduzioni	cartella i18n, non controllata da noi	**non verificato**

01 **Ripeti il metodo per ogni lezione.**
Leggi docs/en.md → scrivi o leggi il codice → eseguilo dalla cartella del repository → salva l'output → continua solo quando sai spiegarlo.

// RICORDA

Un corso gratis funziona solo se lo apri davvero: **una lezione al giorno**. Repository: **github.com/rohitg00/ai-engineering-from-scratch** (autore rohitg00). Stelle, ore e numero di lezioni cambiano: la pagina è la fonte aggiornata.